

STM32 毕设项目 — 软件需求生成模板 v6

工作流：

1. 你在本模板中填写「项目输入」

2. 整份文档发给 Claude（对话）

3. Claude 输出一份或两份**软件需求文档**（.md 文件）

4. 你把对应的需求文档发给 **Claude Code**，Claude Code 在项目源码中执行开发

你只需要填写下面的「项目输入」部分。框架上下文已内嵌在文档末尾，Claude 会自动参考。

项目输入（每次新题目填写这里）

1. 任务需求【必填】

1.1 本课题的研究内容

本课题旨在设计并实现一套基于物联网与嵌入式技术的两轮电动车智能防盗与定位系统。系统针对电动车盗窃案件频发、追踪困难的问题，通过集成多传感器监测电路、主控与无线通信电路、定位模块以及远程交互平台，构建一个实时、可靠的安全防护网络。选用高灵敏度震动传感器、定位模块和无线通信模块；设计低功耗的主控电路与电源管理策略；开发基于多传感器信息融合的智能防盗判定算法，准确区分正常碰触与异常盗窃行为；编写定位追踪与远程通信程序，实现车辆位置的实时上报、历史轨迹查询及远程控制。系统需完成硬件制作、嵌入式软件开发和手机 APP/小程序的原型设计，并进行实地功能与性能测试。设计必须注意作品的实用性和性价比，同时考虑节能、环境、社会、法律等非技术因素。设计 STM32 系列单片机主控电路、传感器监测电路（震动传感器、位移检测）、定位与通信电路、本地声光报警电路、人机交互电路及电源管理电路等，同时设计相应的智能防盗判定算法、定位数据解析程序、云端通信协议及远程交互逻辑，完成一款两轮电动车智能防盗与定位系统的软硬件设计、制作、调试、测试。

具体功能及指标要求如下：

- （1）监测与定位通道：具备震动监测、位移监测、定位，实现多维度状态感知与位置获取；
- （2）定位精度与功耗：室外定位误差≤10 米；系统支持静止休眠模式，在无异常时进入低功耗状态以延长待机时间；
- （3）防盗响应性能：震动、位移等异常触发后，本地声光报警响应，远程报警（推送至手机 APP）响应，防盗误触发率≤10%，响应时间≤5s；
- （4）人机与远程交互：支持手机 APP 远程查看车辆实时位置、回放历史运动轨迹，并可远程启动/关闭防盗功能；设备具备本地显示屏与按键，可查看当前位置坐标、信号强度及系统工作状态；

2. APP 通信变量布局【选填，有 APP 通信则必填】

基于框架的 appcom 模块，规划本项目的变量分配。
注意：TX[0]~[3]（帧0x09）不会到达 APP，APP 可见的 TX 起始索引为 [4]。
不填则由 Claude 根据功能自动分配。

TX (STM32 → APP)

索引	有效宽度	变量含义	数据类型
[4]	4B		
[5]	4B		
[6]	1B		
[7]	1B		
[8]	4B		
[9]	4B		
[10]	1B		
[11]	1B		
[12]	4B		
[13]	4B		
[14]	1B		
[15]	1B		

RX (APP → STM32)

索引	有效宽度	变量含义	数据类型
[0]	4B		
[1]	4B		
[2]	1B		
[3]	1B		
[4]	4B		
[5]	4B		
[6]	1B		
[7]	1B		

3. APP 需求【选填，有 APP 功能则建议填写】

如果项目包含手机 APP 功能，在此填写 APP 相关信息。留空则 Claude 根据任务需求自动设计。

- 项目名称（显示在 APP 主页顶部）：**两轮电动车智能防盗与定位系统设计**
- 项目作者（显示在 APP 主页顶部）：2200810602陈敏敏

以下为可选补充，留空则由 Claude 自动设计：

APP端需要额外处理stm32发来的原始GPS数据，来完成实时位置查看，历史运动轨迹回放功能。原则是实现功能即可，方案任意，简单优先

4. 补充说明【选填】

声光报警通过板载的蜂鸣器和LED灯实现，项目的driver层已实现相关API。OLED上显示当前GPS信息：GPS状态，GPS经纬度坐标（变量部分使用大字体，如：经纬度的具体变量值），休眠倒计时（30s）。stm32进入休眠后通过任意KEY按键按下唤醒。上述功能API均已实现。

GPS信息更新周期1s。对于实时位置，历史运动轨迹回放等功能在上位机APP端实现，stm32只负责周期发送gps数据，不做gps数据处理。

对于防盗响应的震动监测功能，当防盗功能开启时，使用陀螺仪的加速度计等信息融合判断是否震动，在stm32端处理，stm32向app提供是否报警标志位。

对于防盗响应的位移监测功能，当防盗功能开启时，位移超过10m，则报警。

—— 以下为框架上下文（不要修改，Claude 自动参考） ——

串口标准通信协议

所有模块间通信均使用统一的 16 字节定长帧格式：

字节位置	值	说明
[0]	0x55	包头 1
[1]	0xAA	包头 2
[2]	0x00~0x24	控制字段（CMD）
[3]~[12]	—	数据域，UTF-8 编码，最多 10 字节，不足补 0x00
[13]	CRC8	对 [0]~[12] 的 CRC8 校验值（ENABLE_CRC=False 时固定为 0x00）
[14]	CNT	包计数，0x00~0xFF 循环递增
[15]	0xFF	包尾

控制字段分配（全局唯一，不可更改）

控制字段值	方向	说明
0x00	STM32 → 其它模块	广播/通用
0x01~0x04	摄像头模块 → STM32	视觉数据

控制字段值	方向	说明
0x05~0x08	串口屏模块 → STM32	触控事件
0x09~0x0C	STM32 → ESP32	appcom TX 帧 (固定不可改)
0x13~0x16	ESP32 → STM32	appcom RX 帧 (固定不可改)
0x17~0x20	ESP32 → APP	BLE 下行
0x21~0x24	APP → ESP32	BLE 上行

⚠ 控制字段是全局协议，任何模块都不能修改自己或其他模块的 CMD 编号。

<stm32_framework_context>

框架概述

基于 STM32F103C8T6，HAL库 + FreeRTOS CMSIS_V2，分层架构。

开发边界：

- **主要工作：**编写 task 层（应用层）代码，调用 driver 层已有 API
- **允许修改 driver 层的情况：**当某个功能仅靠现有 driver API 无法实现，但通过修改/扩展 driver 层可以实现时，允许修改 driver 层。修改时必须遵守 driver 层的接口规范和命名规范，并在功能实现文档中明确标注修改了哪个 driver 的哪些内容
- **不可修改：**stdlib 层、HAL 层

分层架构与调用规则

task层 (userApp)	← 主要工作区，开发应用逻辑
func层 (userFunc)	← 可选，跨driver逻辑整合（如appcom已实现）
driver层 (userDriver)	← 已完成，现有API不足时允许修改/扩展
stdlib层 (userLib)	← 已完成，不可修改
HAL层 (Core/)	← CubemX生成，不可修改

task层可调用：stdlib层、driver层、func层。禁止直接调用 HAL。

文件结构

MDK-ARM/		
├─ userLib/	# stdlib层（已完成）	
├─ userDriver/	# driver层（按项目提供）	
├─ userFunc/	# func层（appcom等已实现）	
└─ userApp/	# task层（开发工作区）	
├─ task_system.c/.h	← 系统任务，不修改	
└─ task_user1.c/.h	← 应用任务（主要开发文件）	

driver 模块自动发现：项目中 `userDriver/` 目录下保留的 `.c/.h` 文件即为本项目全部可用的 driver 模块（项目负责人已删除无关模块）。开发前先浏览 `userDriver/` 目录，逐个阅读每个 driver 的 `.h` 文件中「向上提供」部分，了解可用的结构体和 API，再开始 task 层设计。

driver 层接口规范

每个 driver 模块提供：

1. 公有信息结构体 `xxxInfo`（或 `xxxInfo[CH_COUNT]`），task 层通过它读取状态
2. `DRIVER_XXX_Init()` — 初始化（在 task 的 Init 中调用一次）
3. `DRIVER_XXX_Update()` — 部分 driver 需要周期调用以更新状态
4. 其他功能函数 `DRIVER_XXX_DoSomething(param)`

task 层开发规范

框架有两个 FreeRTOS 任务：

- **task_system**：基础周期 **10ms**，负责 stdlib 状态机维护和 appcom 通信更新。**不需要修改**此文件。
- **task_user1**：基础周期 **2ms**，应用业务逻辑的主要开发文件，通过此文件完成项目功能。

每个任务文件必须包含：

1. 公有信息结构体 `xxxTaskInfo_t + extern xxxTaskInfo_t xxxTaskInfo`
2. 初始化函数 `xxxTaskInit(void)`
3. 更新函数 `xxxTaskUpdata(void *argument)` — FreeRTOS 任务入口

标准模板：

```
// task_user1.h
typedef struct {
    uint32_t taskCnt;
    // 任务相关状态数据...
} user1TaskInfo_t;
extern user1TaskInfo_t user1TaskInfo;
void user1TaskInit(void);
void user1TaskUpdata(void *argument);

// task_user1.c
user1TaskInfo_t user1TaskInfo;

void user1TaskInit(void) {
    DRIVER_XXX_Init();
    // 初始化任务内部状态
}

void user1TaskUpdata(void *argument) {
    user1TaskInit();
    for(;;) {
        user1TaskInfo.taskCnt++;
        DRIVER_XXX_Update(); // 需要周期更新的driver
    }
}
```

```
// 通过 taskCnt 取余获得更长的子周期（基础周期 2ms）
if(user1TaskInfo.taskCnt % 5 == 0) {    // 10ms
    // 10ms周期逻辑
}
if(user1TaskInfo.taskCnt % 50 == 0) {    // 100ms
    // 100ms周期逻辑
}
if(user1TaskInfo.taskCnt % 250 == 0) {    // 500ms
    // 500ms周期逻辑
}
if(user1TaskInfo.taskCnt % 500 == 0) {    // 1s
    // 1s周期逻辑
}

osDelay(2); // 基础调度周期 2ms
}
```

APP 通信模块（func_appcom，已实现）

⚠ 严禁修改 func_appcom.c/h ⚠

func_appcom 中的帧 CMD 编号和帧数量是框架与 ESP32 固件的**固定约定**，修改会导致 BLE 断连。以下参数不可更改：

STM32 TX 帧号（固定）：0x09，0x0A，0x0B，0x0C（共4帧）
STM32 RX 帧号（固定）：0x13，0x14，0x15，0x16（共4帧）
帧数量：4 TX + 4 RX（固定）

STM32 → APP 链路（TX 方向）

ESP32 对 STM32 TX 帧的转发规则：

STM32 TX CMD	ESP32 行为	APP RX CMD	说明
0x09	ESP32 截留，不转发给 APP	0x17 由 ESP32 自己填充发送	STM32 TX[0]~[3] 对 APP 不可见
0x0A	ESP32 转发	0x18	STM32 TX[4]~[7] → APP 可见
0x0B	ESP32 转发	0x19	STM32 TX[8]~[11] → APP 可见
0x0C	ESP32 转发	0x1A	STM32 TX[12]~[15] → APP 可见

⚠ 关键约束：`remoteVar_TX[0]~[3]`（帧 0x09）的数据不会到达 APP。发送给 APP 的变量必须使用 TX[4]~[15]。

TX[0]~[3] 仍可用于 STM32 与 ESP32 之间的私有通信，但不要用它们向 APP 传递数据。

APP → STM32 链路 (RX 方向)

APP 发送的所有帧由 ESP32 全部转发给 STM32，无截留：

APP TX CMD	ESP32 行为	STM32 RX CMD	说明
0x21	ESP32 全部转发	0x13	→ STM32 RX[0]~[3]
0x22	ESP32 全部转发	0x14	→ STM32 RX[4]~[7]
0x23	ESP32 全部转发	0x15	→ STM32 RX[8]~[11]
0x24	ESP32 全部转发	0x16	→ STM32 RX[12]~[15]

task 层只通过 `remoteVar_TX[0]~[15]` / `remoteVar_RX[0]~[15]` 读写数据，不需要知道底层帧号。

通信链路：STM32 → USART1 → ESP32 → BLE → APP（双向）

task 层只操作 `remoteInfo` 结构体，`FUNC_APPCOM_UPDATA()` 由 `task_system` 自动调用。

变量池：`remoteVar_TX[16]` / `remoteVar_RX[16]`，每个元素为 `remoteVar_t`（4字节联合体）：

```
typedef union {
    uint8_t  raw[4];           /* 字节视图，与下方三个成员共享同一块4字节内存 */
    uint32_t var_uint32;
    int32_t  var_int32;
    float    var_float;
} remoteVar_t;
```

⚠ 禁止在 union 内部嵌套 struct，否则 struct 会扩展为12字节，破坏4字节内存共享，导致 raw 与 var_float 读写错位。

索引与帧槽位映射（TX/RX 各16个）：

- [0][1][4][5][8][9][12][13] → 4字节槽位（float/uint32/int32 均可）
- [2][3][6][7][10][11][14][15] → 1字节槽位（仅 var_uint32 低8位有效）

字节序约定

方向	APP端	STM32端	处理位置
APP → STM32 (RX)	大端序发送4字节字段	func_appcom 内 <code>__APPCOM_SwapU32()</code> 已自动翻转为小端	func_appcom.c（已实现，勿改）
STM32 → APP (TX)	APP自行解析，无需关心	直接赋值，无字节序问题	无需处理

task层字节序规则（必须遵守）：

- 读取 RX：直接使用 `var_float` / `var_uint32`，func_appcom 已完成翻转，禁止在 task 层再做任何字节序处理
- 写入 TX：直接赋值，禁止在 task 层做任何字节序处理

使用方式：

```
/* TX: 发送给APP的数据必须用 TX[4]~[15] (TX[0]~[3] 不会到达APP)
 * 直接赋值即可，无需处理字节序 */
remoteInfo.remoteVar_TX[4].var_float    = currentTemp;
remoteInfo.remoteVar_TX[6].var_uint32   = (uint32_t)mode;

/* RX: 接收APP的数据用 RX[0]~[15] (全部可用)
 * func_appcom 已完成大端→小端翻转，直接读取即可，禁止再翻转 */
float    target = remoteInfo.remoteVar_RX[0].var_float;
uint8_t  appKey = (uint8_t)(remoteInfo.remoteVar_RX[2].var_uint32 & 0xFF);
```

编码约束

约束项	规则
内存分配	禁止 malloc，全部静态分配
注释语言	中文
私有定义	写在 .c 文件中
公有定义	写在 .h 文件中
命名风格	task层：小驼峰 (user1TaskInit) ； 结构体：xxxTaskInfo_t; 宏：MODULE_CONSTANT
额外功能	不实现需求未提及的功能

</stm32_framework_context>

<app_framework_context>

APP 框架概述

Android 原生项目，Jetpack Compose + Material Design 3，BLE 通信层已完成。

开发边界：

- **主要工作**：修改 MainActivity.kt 中的 HomeScreen() 函数，实现 UI 展示和控制交互
- **不可修改**：BleProtocol.kt、BleConnectionManager.kt、BleAutoConnector.kt 等通信底层文件

文件结构

```
app/src/main/java/com/example/demo_1/
├─ MainActivity.kt          # 主页面（HomeScreen()）← 主要开发文件
├─ BleScanActivity.kt      # BLE 扫描（不修改）
├─ SecondActivity.kt       # 设置页（不修改）
├─ ThirdActivity.kt        # 调试终端（不修改）
├─ App.kt                  # Application 入口（不修改）
├─ AppBottomNavigation.kt  # 底部导航栏（不修改）
├─ BleProtocol.kt          # 帧解析/构建（不修改）
├─ BleConnectionManager.kt # GATT 连接/读写（不修改）
├─ BleAutoConnector.kt     # 后台自动连接（不修改）
├─ userConfig.kt           # 全局配置（不修改）
└─ ui/theme/               # 主题（不修改）
```

数据读取方式（RX：→ APP）

APP 接收数据通过 `BleProtocol.rxFrames` 访问。

⚠ APP RX 0x17 帧由 ESP32 专属发送，不来自 STM32。APP 能读到的 STM32 数据从 0x18 帧开始。

来源	APP RX CMD	对应 STM32 remoteVar_TX 索引	说明
ESP32 专属	0x17	无对应 (TX[0]~[3] 不转发)	ESP32 自行填充，内容由 ESP32 固件决定
STM32 TX 0x0A	0x18	TX[4]~[7]	STM32 数据，APP 可读
STM32 TX 0x0B	0x19	TX[8]~[11]	STM32 数据，APP 可读
STM32 TX 0x0C	0x1A	TX[12]~[15]	STM32 数据，APP 可读

每帧4个槽位：`var4b1` (4B)、`var4b2` (4B)、`var1b1` (1B)、`var1b2` (1B)

```
// 读取 STM32 数据（从 0x18 开始，不要用 0x17 读 STM32 数据）
val frame18 = BleProtocol.rxFrames[0x18]
val temperature = frame18?.var4b1 ?: 0 // 对应 STM32 TX[4]
val targetTemp  = frame18?.var4b2 ?: 0 // 对应 STM32 TX[5]
val mode        = frame18?.var1b1 ?: 0 // 对应 STM32 TX[6]
val status      = frame18?.var1b2 ?: 0 // 对应 STM32 TX[7]

// 0x17 帧是 ESP32 专属数据，不是 STM32 发送的
val frame17 = BleProtocol.rxFrames[0x17] // ESP32 专属帧
```

`rxFrames` 是 `mutableStateOf`，Compose UI 读取时自动触发重组。

数据发送方式（TX： APP → STM32）

APP端以**大端序**发送4字节 float/uint32 字段（Android ByteBuffer 默认即大端序，无需额外设置）。STM32端 func_appcom 已通过 `__APPCOM_SwapU32()` 自动翻转为小端序，APP端无需关心STM32的字节序。1字节字段无字节序问题，直接填写即可。

APP TX CMD	STM32 RX CMD	对应 STM32 remoteVar_RX 索引	说明
0x21	0x13	RX[0]~[3]	全部转发
0x22	0x14	RX[4]~[7]	全部转发
0x23	0x15	RX[8]~[11]	全部转发
0x24	0x16	RX[12]~[15]	全部转发

```
// 构建并发送帧
val frame = BleProtocol.buildTxFrame(
    cmd      = 0x21,
    var4_1 = 0,      // 4字节参数1
    var4_2 = 0,      // 4字节参数2
    var1_1 = 0,      // 1字节参数1
    var1_2 = 0       // 1字节参数2
)
val ok = BleConnectionManager.writeCharacteristic(
    serviceUuid      = UserConfig.esp32_service_1_uuid,
    characteristicUuid = UserConfig.esp32_service_1_characteristic_1_uuid,
    value             = frame
)
if (ok) BleConnectionManager.recordOutgoingMessage(
    characteristicUuid = UserConfig.esp32_service_1_characteristic_1_uuid,
    value = frame
)
```

可复用 UI 组件

```
// 分区卡片
HomeSection(title = "系统状态") {
    // 内部放 Row / Text 等
}

// 数据行
ProtoFieldRow(label1 = "温度", value1 = "38.5°C",
               label2 = "目标", value2 = "42°C")
```

APP 端变量与 STM32 端 remoteVar 的索引映射

STM32 TX → APP RX (仅 TX[4]~[15] 对 APP 可见)

remoteVar_TX 索引	APP RX CMD	槽位	有效宽度	APP 可见?
[0]~[3]	—	—	—	✗ 不转发 (ESP32截留)
[4]	0x18	var4b1	4B	✓
[5]	0x18	var4b2	4B	✓
[6]	0x18	var1b1	1B	✓
[7]	0x18	var1b2	1B	✓
[8]	0x19	var4b1	4B	✓
[9]	0x19	var4b2	4B	✓
[10]	0x19	var1b1	1B	✓
[11]	0x19	var1b2	1B	✓
[12]	0x1A	var4b1	4B	✓
[13]	0x1A	var4b2	4B	✓
[14]	0x1A	var1b1	1B	✓
[15]	0x1A	var1b2	1B	✓

APP TX → STM32 RX (全部转发)

remoteVar_RX 索引	APP TX CMD	槽位	有效宽度
[0]	0x21	var4_1	4B
[1]	0x21	var4_2	4B
[2]	0x21	var1_1	1B
[3]	0x21	var1_2	1B
[4]	0x22	var4_1	4B
[5]	0x22	var4_2	4B
[6]	0x22	var1_1	1B
[7]	0x22	var1_2	1B
[8]~[11]	0x23	同上规律	—
[12]~[15]	0x24	同上规律	—

==== Claude 输出规则（Claude 读到此处后按以下规则输出）====

收到本模板后，Claude 输出 **一份或两份** 软件需求文档（.md 文件），直接发送给对应的 Claude Code 执行：

- **始终输出**：STM32 软件需求文档（给 STM32 项目目录下的 Claude Code）
- **如果任务需求包含 APP 功能**：额外输出 APP 软件需求文档（给 APP 项目目录下的 Claude Code）

文档一：STM32 软件需求文档

一、项目概述

简要说明项目做什么、核心功能有哪些。

二、开发前置步骤

指示 Claude Code：

1. 浏览 MDK-ARM/userDriver/ 目录，阅读每个 driver 的 .h 文件中「向上提供」部分
2. 汇总所有可用的结构体和 API
3. 以此为基础进行后续开发

二（附）、禁止修改的文件清单

必须在 STM32 需求文档中明确列出以下禁止修改的文件，并解释原因：

- **func_appcom.c / func_appcom.h** — 帧 CMD 编号（TX: 0x09~0x0C, RX: 0x13~0x16）和帧数量（4+4）是框架与 ESP32 固件的固定约定，修改会导致 BLE 断连
- **task_system.c / task_system.h** — 系统任务，框架已实现
- **userLib/ 下所有文件** — stdlib 层，框架已实现
- **Core/ 下所有文件** — HAL 层，CubeMX 生成

同时必须包含以下注意事项：

- **TX[0]~[3]（帧0x09）不会到达 APP**，发送给 APP 的变量必须使用 TX[4]~[15]
- 不要整体清零 remoteVar_TX（只写需要的索引，未使用的保持原值）
- 不要在代码中出现 0x17~0x1A、0x21~0x24 等 APP 端帧号（STM32 端不需要知道这些）

三、task 层设计

1. **文件规划** — 需要哪些 task 文件，各自职责（默认只用 task_user1）
2. **公有结构体设计** — user1TaskInfo_t 包含哪些字段，逐一说明用途
3. **Init 函数设计** — 调用哪些 DRIVER_XXX_Init()，初始化哪些状态
4. **Udata 循环设计** — 各功能的执行周期（基于 2ms 基础周期的 taskCnt 取余）、状态机设计、逻辑流程

四、APP 通信变量布局（如有）

列出 remoteVar_TX/RX 的完整分配表，说明每个索引对应的变量含义和数据类型。
TX 部分只列 [4]~[15]（对 APP 可见的索引），TX[0]~[3] 如需用于 ESP32 私有通信则单独标注。

五、功能详细需求

逐个功能模块详细描述：

- 功能描述
- 实现逻辑（状态机、算法、判断条件等）
- 涉及的 driver API 调用
- 关键参数和阈值

六、driver 层修改需求（如需要）

如果根据需求分析，现有 driver API 不足以实现某些功能，在此列出：

- 需要修改/新增的 driver 模块
- 修改内容和原因
- 必须遵守 driver 层的接口规范和命名规范

七、输出物要求

指示 Claude Code 完成开发后，必须输出：

1. 代码文件

- task_user1.c / task_user1.h（以及其他需要的 task 文件）
- 如果修改了 driver 层，输出修改后的 driver 文件并用注释标注修改点

2. 功能实现文档（implementation_report.md），包含：

(a) 功能实现状态清单

序号	功能描述	状态	说明
1		✅ / ⚠️ / ❌	

状态分三级：✅ 已实现、⚠️ 部分实现（说明缺什么）、❌ 未实现（说明原因）

(b) driver 层修改记录（如有修改）

文件	修改类型	修改内容	原因
----	------	------	----

(c) 已实现功能的测试步骤

对每个已实现的功能，给出具体的测试方法和预期结果，让开发者可以逐项验证。

文档二：APP 软件需求文档（仅当任务需求包含 APP 功能时输出）

一、项目概述

简要说明 APP 需要实现哪些界面和交互功能。

二、开发边界

指示 Claude Code：

- 修改 MainActivity.kt 中的 HomeScreen() 函数和 userConfig.kt 中的项目名称以及项目作者信息
- 不修改 BleProtocol.kt、BleConnectionManager.kt 等通信底层文件
- UI 使用 Jetpack Compose + Material Design 3
- 可复用 HomeSection() 和 ProtoFieldRow() 组件

三、通信变量映射表

列出 APP 端需要读/写的所有变量，包含完整的 CMD 和槽位映射。

⚠ APP 接收表中必须注明：0x17 帧由 ESP32 专属发送，不来自 STM32。STM32 数据从 0x18 帧开始。

APP 接收（显示用）：

数据含义	APP RX CMD	槽位	数据类型	对应 STM32 remoteVar_TX 索引
------	------------	----	------	--------------------------

APP 发送（控制用）：

操作含义	APP TX CMD	槽位	数据类型	对应 STM32 remoteVar_RX 索引
------	------------	----	------	--------------------------

并给出具体的读取和发送代码示例。

四、UI 界面设计

强制要求：HomeScreen() 顶部必须包含一个标题区域，显示以下信息（从模板第3节「APP 需求」中获取）：

- 项目名称（大字体居中显示）
- 项目作者（小字体，显示在项目名称下方）

如果用户未填写项目名称/作者，则使用任务需求中的项目名称，作者显示为空。

UI 设计原则（必须遵守）：

1. STM32 → APP 的数据（接收） = 文本显示
 - 每个接收变量显示为一行：属性描述：属性值
 - 用 HomeSection 分区，用 Text 或 ProtoFieldRow 显示
 - 示例：当前水温：38.5°C、播放状态：播放中、音量：80
2. APP → STM32 的控制（发送） = 按钮
 - 所有控制操作一律用 Button 实现，按钮上写操作描述

- 示例：[音量+] [音量-] [播放/暂停] [下一曲] [模式切换] [定时:15分]
- 需要设定具体数值的功能（如音量、亮度、定时），用多个按钮代替（如 [音量+] [音量-]，或 [亮度20%] [亮度50%] [亮度80%] [亮度100%]）

3. 禁止使用的复杂控件

- 不要使用 Slider（滑块/进度条）
- 不要使用 SeekBar
- 不要使用自定义拖动控件
- 除非用户在模板第3节「APP 需求」或第4节「补充说明」中明确要求使用这些控件

标题区域之后，用 HomeSection 划分功能区域：

- 每个区域包含哪些显示元素和控制按钮
- 数据显示的格式（如温度保留1位小数、状态用中文文字等）
- 按钮的点击逻辑（调用 buildTxFrame + writeCharacteristic 发送什么数据）

五、交互逻辑

描述 APP 端的业务逻辑：

- 数据刷新方式（Compose 自动重组，无需手动刷新）
- 按钮状态管理（如开关机按钮的文字/颜色切换）
- 异常状态的 UI 表现（故障码展示、颜色变化等）
- 所有控制通过按钮点击触发，点击后调用 sendFrame 发送对应数据

六、输出物要求

指示 Claude Code 完成开发后，必须输出：

1. 代码文件

- 修改后的 MainActivity.kt（标注修改区域）

2. 功能实现文档（app_implementation_report.md），包含：

(a) 功能实现状态清单

序号	功能描述	状态	说明
1		✅ / ⚠️ / ❌	

(b) 已实现功能的测试步骤

对每个 APP 功能，给出具体的测试方法和预期结果。

输出约束

- **func_appcom 不可修改（最高优先级）**：STM32 需求文档中必须明确禁止 Claude Code 修改 func_appcom.c/h。帧 CMD 编号（TX: 0x09~0x0C, RX: 0x13~0x16）和帧数量（4TX+4RX）是框架与 ESP32

固件的固定协议，修改会导致 BLE 断连。STM32 需求文档中不得出现 0x17~0x1A、0x21~0x24 等 APP 端帧号

- **TX[0]~[3] 对 APP 不可见**：帧 0x09 被 ESP32 截留不转发。Claude 分配 APP 通信变量时，TX 只能使用 [4]~[15]。TX[0]~[3] 可用于 STM32-ESP32 私有通信但不能用于 APP 显示
- **0x17 帧为 ESP32 专属**：APP 端 0x17 帧的数据由 ESP32 自行填充，不来自 STM32。APP 需求文档中必须注明这一点，避免 APP 端 Claude Code 从 0x17 帧读取 STM32 数据
- **不要整体清零 remoteVar_TX**：只写需要的索引，未使用的保持原值。整体清零可能导致未使用帧发送异常数据
- **通信一致性**：Claude 必须先确定 remoteVar_TX/RX 的完整统一布局表，再分别写入 STM32 和 APP 两份文档。两份文档中同一个 remoteVar 索引的变量含义、数据类型、方向必须完全一致
- **APP→STM32 通信方案**：采用直接槽位映射，即每个功能固定占一个 remoteVar_RX 索引（如 RX[0]=设定音量, RX[2]=播放暂停按键），STM32 端直接读取对应索引即可，不使用"指令码+参数"方案
- **STM32 需求文档必须包含验证清单**，至少包括：确认 func_appcom 无改动、确认代码中无 APP 端帧号、确认 remoteVar 索引无越界、确认 TX 发送给 APP 的变量未使用 [0]~[3]
- 需求文档中引用框架规范时，直接写明规则（如命名风格、分层约束、API 调用方式），不要让 Claude Code 再去找本模板
- 不要在 STM32 需求文档中生成 CubeMX 配置
- 不要指示 STM32 Claude Code 修改 task_system、stdlib 层、HAL 层
- 不要指示 APP Claude Code 修改 BleProtocol.kt、BleConnectionManager.kt 等通信底层文件
- 两份需求文档应当各自包含——对应的 Claude Code 拿到它 + 项目源码就能完成开发，不需要其他文档
- APP 需求文档中必须包含完整的 CMD 和槽位映射，并注明 0x17 帧为 ESP32 专属
- APP 主页面必须包含项目名称和项目作者显示